

Практическая работа «Автоматизация управления базой данных»

Цель работы: научиться создавать с помощью Access модули для автоматизации работы приложения

Теоретическая часть

Общие сведения об автоматизации приложения

Автоматизировать работу приложения базы данных можно с помощью модулей и макросов. Модули являются более универсальным средством, чем макросы, и рекомендуются Microsoft для автоматизации при создании новых приложений. Перечислим вопросы автоматизации приложения, которые нельзя решить с помощью макросов или целесообразно решать с помощью модулей:

- Обработка ошибок в приложении.
- Создание пользовательских функций.
- Создание новых объектов (таблиц, форм, отчетов, запросов) во время работы приложения.
- Использование системных функций Windows.
- Необходимость обработки отдельных записей.

Модуль – это набор объявлений и процедур на языке VBA, собранных в одну программную единицу. Существует два основных типа модулей: стандартные модули и модули класса.

В стандартных модулях содержатся общие процедуры и функции, не связанные ни с каким объектом, а также часто используемые процедуры. Список стандартных модулей базы данных отображается на вкладке **Модули** в окне базы данных. Новый стандартный модуль создается аналогично остальным объектам Access (формам, отчетам и т.д.) путем выбора кнопки **Создать** в окне базы данных при выбранных объектах **Модули**.

Модули форм и модули отчетов являются модулями класса, связанными с определенной формой или отчетом. Они часто содержат процедуры обработки событий, запускаемых в ответ на событие в форме или отчете. Процедуры обработки событий используются для управления поведением формы или отчета и их откликом на события, например такие, как нажатие кнопки. При создании первой процедуры обработки события для формы или отчета автоматически создается связанный с ней модуль формы или отчета. В процедурах модулей форм и отчетов могут содержаться вызовы процедур, добавленных в стандартные модули.

Модуль класса создается путем щелчка по кнопке построителя для одного из свойств событий формы, отчета или элемента управления с последующим выбором пункта меню **Программа**. Для открытия модуля класса формы необходимо в режиме конструктора щелкнуть по кнопке  главной кнопочной панели.

Каждый модуль состоит из раздела описаний (Declaration Section) и одной или нескольких процедур или функций. В разделе описаний модуля содержатся описания используемых в модуле констант и переменных.

События в формах и отчетах

Событие – это определенное действие, которое происходит над или возникает в определенном объекте. Обычно события возникают вследствие действий пользователя.

С помощью процедур обработки события или макроса возможно определение собственных реакций на события, происходящие в форме, отчете или элементе управления. Чтобы в ответ на событие запустить модуль, следует открыть окно свойств для формы, отчета или элемента управления, найти соответствующее событию свойство и установить в качестве его значения вызов соответствующей процедуры.

Рассмотрим наиболее часто используемые события в формах и элементах управления. При описании событий в скобках будут перечисляться объекты, для которых возможно это событие.

События данных возникают при вводе, удалении или изменении данных в форме или элементе управления, а также при перемещении фокуса с одной записи на другую.

После подтверждения Del (AfterDelConfirm) (формы). Возникает после подтверждения пользователем удаления записей и фактического удаления записей или после отмены удаления.

После вставки (AfterInsert) (формы). Возникает после добавления новой записи в базу данных.

После обновления (AfterUpdate) (формы, элементы управления). Возникает после обновления данных в элементе управления или записи при потере фокуса элементом управления или записью. Событие возникает для новых и существующих записей.

До подтверждения Del (BeforeDelConfirm) (формы). Возникает после удаления одной или нескольких записей, но до открытия диалогового окна Microsoft Access с приглашением подтвердить или отменить удаление. Данное событие возникает после события Удаление (*Delete*).

До вставки (BeforeInsert) (формы). Возникает при вводе первого символа в новую запись, но до добавления записи в базу данных.

До обновления (BeforeUpdate) (формы, элементы управления). Возникает при потере фокуса элементом. Событие возникает для новых и существующих записей.

Изменение (Change) (элементы управления). Возникает при изменении содержимого в поле или в поле со списком.

Удаление (Delete) (формы). Возникает при удалении записи пользователем, но до подтверждения удаления и до фактического удаления записи.

Отсутствие в списке (NotInList). Возникает при вводе в поле со списком значения, отсутствующего в списке поля со списком.

При обновлении (Updated) (элементы управления). Возникает при изменении данных в объекте.

События клавиатуры возникают при вводе с клавиатуры, а также при передаче нажатий клавиш с помощью инструкции SendKeys.

Клавиша вниз (KeyDown) (формы, элементы управления). Возникает при нажатии пользователем любой клавиши на клавиатуре в тот момент, когда элемент управления или форма имеют фокус. Форма может получить фокус только в том случае, когда все ее видимые элементы управления недоступны или когда в форме нет элементов управления. Если пользователь нажимает и удерживает клавишу, то событие *Клавиша вниз (KeyDown)* возникает многократно.

Нажатие клавиши (KeyPress) (формы, элементы управления). Возникает, если пользователь нажимает и отпускает клавишу в момент, когда элемент управления или форма имеют фокус. Если пользователь нажимает и удерживает клавишу, то событие *Нажатие клавиши* возникает многократно.

Клавиша вверх (KeyUp) (формы, элементы управления). Возникает, если пользователь отпускает нажатую клавишу в момент, когда элемент управления или форма имеют фокус. Если пользователь нажимает и удерживает клавишу, то событие *Клавиша вверх* возникает после всех событий *Клавиша вниз* и *Нажатие клавиши*.

События мыши возникают при действиях с мышью.

Нажатие кнопки (Click) (формы, элементы управления). Для элемента управления это событие возникает, когда пользователь нажимает и быстро отпускает левую кнопку мыши при указателе, установленном на элементе управления. Для формы это событие возникает, когда пользователь выбирает при помощи мыши область выделения записи, а также область формы вне раздела или элемента управления.

Двойное нажатие кнопки (DbClick) (формы, элементы управления). Для элемента управления это событие возникает, когда пользователь дважды быстро нажимает и отпускает левую кнопку мыши при указателе, установленном на элементе управления или присоединенной к нему надписи. Для формы это событие возникает, когда пользователь дважды быстро нажимает и отпускает кнопку мыши при указателе, установленном в области выделения записи или в пустой области формы.

Кнопка вниз (MouseDown) (формы, элементы управления). Возникает в момент нажатия кнопки мыши при указателе, установленном на форме или на элементе управления.

Перемещение указателя (MouseMove) (формы, элементы управления). Возникает при перемещении указателя по форме или элементу управления.

Кнопка вверх (MouseUp) (формы, элементы управления). Возникает в момент отпускания нажатой кнопки мыши при указателе, установленном на форме или на элементе управления.

События фокуса возникают, когда форма или элемент управления теряют или получают фокус, а также в момент, когда форма или отчет становятся активным или неактивным окном.

Включение (Activate) (формы, отчеты). Возникает, когда окно формы или отчета становится активным.

Отключение (Deactivate) (формы, отчеты). Возникает при выполнении действия, в результате которого активным должно стать другое окно Microsoft Access, но до того, как это окно действительно станет активным.

Вход (Enter) (элементы управления). Возникает перед фактическим получением фокуса элементом управления от другого элемента управления в той же форме или при открытии формы. Данное событие возникает до события *Получение фокуса (GotFocus)*.

Выход (Exit) (элементы управления). Возникает непосредственно перед переводом фокуса на другой элемент управления в той же форме. Данное событие возникает до события *Потеря фокуса (LostFocus)*.

Получение фокуса (GotFocus) (формы, элементы управления). Возникает при получении фокуса элементом управления или формой, не имеющей активных или доступных элементов управления. Форма может получить фокус только в том случае, когда все ее видимые элементы управления недоступны или когда в форме нет элементов управления.

Потеря фокуса (LostFocus) (формы, элементы управления). Возникает при потере фокуса элементом управления или формой. Форма может получить фокус только в том случае, когда все ее видимые элементы управления недоступны или когда в форме нет элементов управления.

События окна возникают при открытии, изменении размеров или закрытии формы или отчета.

Закрытие (Close) (формы, отчеты). Возникает при закрытии формы или отчета.

Загрузка (Load) (формы). Возникает при открытии формы и выводе в ней записей. Данное событие возникает после события *Открытие (Open)*.

Открытие (Open) (формы, отчеты). Возникает при открытии формы, но до вывода первой записи. При открытии отчета – до начала печати отчета.

Изменение размера (Resize) (формы). Возникает при изменении размеров и первом открытии формы.

Выгрузка (Unload) (формы). Возникает при закрытии формы и выгрузке ее записей, но до удаления формы с экрана. Данное событие возникает до события *Закрытие (Close)*.

При обработке ошибок можно использовать событие *Ошибка (Error)* (формы, отчеты). Возникает при возникновении ошибки выполнения Microsoft Access, когда фокус находится в форме или отчете. В число таких ошибок включаются ошибки ядра базы данных Microsoft Jet, но не включаются ошибки выполнения Visual Basic.

Краткие сведения об объектах приложения

Microsoft Access включает следующие основные подсистемы: ядро приложения Application Engine, обеспечивающее пользовательский интерфейс приложения и ядро базы данных Microsoft Jet (Jet DBEngine), обеспечивающее управление хранением и доступом к данным. Каждая система имеет свою

иерархию объектов. Рассмотрим наиболее часто используемые объекты и семейства объектов (коллекции) ядра приложения.

Семейство Forms содержит все формы, открытые в данный момент в базе данных Microsoft Access. Семейство Forms используется в программах VBA или в выражениях для ссылок на формы, открытые в данный момент.

Объект Form представляет конкретную форму Microsoft Access и является компонентом семейства Forms, в которое входят все текущие открытые формы. В рамках семейства Forms отдельные формы нумеруются с помощью индекса, начинающегося с нуля. Допускаются ссылки на объект Form в семействе Forms либо по имени соответствующей формы, либо по ее индексу в семействе. Например *Forms![Микросхемы]*, *Forms("Микросхемы")*, *Forms(0)*. При ссылках на определенную форму рекомендуется применять ссылки по имени, поскольку индекс формы в семействе может измениться. Имена, содержащие пробелы и другие специальные символы, следует заключать в квадратные скобки.

Каждый объект Form содержит семейство Controls, включающее все элементы управления в форме (объекты Control). При ссылках на конкретные элементы управления в форме допускаются как явные, так и неявные ссылки на семейство Controls. Программа с неявными ссылками выполняется быстрее. Пример неявной ссылки: *Forms!Микросхемы!Обозначение*. Пример явной ссылки: *Forms!Микросхемы.Controls!Обозначение*. При ссылке в модуле формы на элементы управления этой же формы лучше (по соображениям скорости выполнения) использовать синтаксис вида *Me!ИмяЭлементаУправления*.

Для работы с объектами используются их методы и свойства. Рассмотрим несколько наиболее часто используемых методов для работы с формами и элементами управления.

Метод Requery обновляет данные, выводящиеся в указанной форме или в элементе управления в активной форме с помощью выполнения повторного запроса к источнику данных формы или элемента управления. Синтаксис:

имяОбъекта.Requery

Аргумент *имяОбъекта* представляет имя объекта Form или Control, определяющий форму или элемент управления, который необходимо изменить.

Метод *Requery* отображает результаты добавления или удаления записей, выполненного после последнего запроса к источнику записей. Метод *Requery* выполняется быстрее, чем макрокоманда *Обновление (Requery)*.

Метод SetFocus переводит фокус на указанную форму или на элемент управления активной формы, а также на поле активного объекта в режиме таблицы. Синтаксис

имяОбъекта.SetFocus

Аргумент имеет тот же смысл, что и в методе *Requery*.

Метод *SetFocus* применяют при необходимости перевести фокус на определенное поле или элемент управления для того, чтобы все вводящиеся

пользователем данные были адресованы этому объекту.

Для считывания значений некоторых свойств элемента управления необходимо, чтобы этот элемент управления имел фокус. Например, для того чтобы считать значение свойства Text поля, необходимо перевести фокус на это поле.

Допускается перемещение фокуса только на видимый элемент управления или форму. Нельзя поместить фокус на элемент управления, если свойство Доступ (Enabled) этого элемента управления имеет значение False. Однако допускается перевод фокуса на элемент управления, у которого свойство Блокировка (Locked) имеет значение True.

Наиболее просто автоматизация работы приложения обеспечивается за счет использования методов объекта DoCmd [1].

1.4 Объект DoCmd

Методы, определенные для объекта DoCmd, позволяют запускать макрокоманды Microsoft Access из программ Visual Basic. Для использования методов объекта DoCmd следует применять следующий синтаксис:

DoCmd.метод [arg1, arg2, ...]

Аргументы синтаксиса методов объекта DoCmd:

Метод – имя одного из методов, поддерживаемых объектом;

arg1, arg2, ... – аргументы указанного метода.

Большинство из методов, определенных для объекта DoCmd, имеют аргументы, некоторые из которых являются обязательными, а другие необязательными. Если необязательный аргумент опущен, то при выполнении макрокоманды подразумевается значение данного аргумента по умолчанию.

Необязательный аргумент посреди списка аргументов разрешается пропустить, однако при этом необходимо ввести запятую, представляющую пропущенный аргумент. Если опускаются один или несколько последних аргументов, вводить запятые вслед за последним указанным аргументом не требуется.

Большинству макрокоманд соответствуют методы DoCmd, имена которых совпадают с английскими именами макрокоманд. Свойства у объекта DoCmd отсутствуют.

Метод Close закрывает объект. Метод имеет следующий синтаксис:

DoCmd.Close [типОбъекта, имяОбъекта], [сохранение]

Аргументы метода:

типОбъекта – одна из следующих встроенных констант: acDefault (значение по умолчанию), acForm (форма), acMacro (макрос), acModule (модуль), acQuery (запрос), acReport (отчет), acTable (таблица);

имяОбъекта – строковое выражение, представляющее допустимое имя объекта, тип которого указан в аргументе *типОбъекта*.

сохранение – одна из встроенных констант: acSaveNo (выход без сохранения), acSavePrompt (подтверждение – значение по умолчанию), acSaveYes (сохранение без подтверждения).

Если оставлены пустыми аргументы *типОбъекта* и *имяОбъекта* (по умолчанию аргументу *типОбъекта* присваивается значение *acDefault*), Microsoft Access закрывает активное окно. Для того чтобы определить аргумент сохранения и оставить аргументы *типОбъекта* и *имяОбъекта* пустыми, необходимо ввести запятые, представляющие аргументы *типОбъекта* и *имяОбъекта*.

Метод *DeleteObject* удаляет объект. Метод имеет следующий синтаксис:

DoCmd.DeleteObject [типОбъекта, имяОбъекта]

Аргументы метода:

типОбъекта – одна из констант аналогично методу *Close*;

имяОбъекта – строковое выражение, представляющее допустимое имя объекта, тип которого указан в аргументе *типОбъекта*.

Если оставлены пустыми аргументы *типОбъекта* и *имяОбъекта*, Microsoft Access удаляет объект, выделенный в окне базы данных.

Метод *FindRecord* позволяет найти запись в соответствии с заданным образцом. Метод имеет следующий синтаксис:

DoCmd.FindRecord образец [, совпадение] [, регистр] [, поиск] [, формат] [, текущееПоле] [, сНачала]

Метод *FindRecord* использует следующие аргументы:

образец – выражение, значением которого является текст, число или дата, задающее образец для поиска;

совпадение – одна из следующих констант: *acAnywhere* (с любой частью поля), *acEntire* (со значением поля) – значение по умолчанию, *acStart* (с начальными символами в значении поля);

регистр – значение *True* (-1) задает поиск с учетом регистра символов, а *False* (0) – поиск без учета регистра. Если оставить данный аргумент пустым, подразумевается значение по умолчанию (*False*).

поиск – одна из следующих встроенных констант: *acDown* (вниз), *acSearchAll* (везде) – значение по умолчанию, *acUp* (вверх);

формат – значение *True* задает поиск с учетом формата полей, а значение *False* – поиск данных в том виде, в котором они сохраняются в базе данных. Значение по умолчанию – *False*.

текущееПоле – одна из следующих встроенных констант: *acAll* (все поля), *acCurrent* (текущее поле) – значение по умолчанию;

сНачала – значение *True* задает начало поиска с первой записи, *False* задает начало поиска с текущей записи. Значение по умолчанию – *True*.

Следующая конструкция обнаружит в текущем поле фамилию «Рост». «Ростов» или «рост» найдены не будут.

DoCmd.FindRecord "Рост", True,,, True

Метод *FindNext* позволяет найти следующую запись, удовлетворяющую аргументам предыдущего метода *FindRecord*. Синтаксис:

DoCmd.FindNext

Данный метод не требует аргументов.

Метод GoToRecord позволяет перейти на определенную запись.
Синтаксис:

DoCmd.GoToRecord [типОбъекта, имяОбъекта] [, запись] [, смещение]

Метод GoToRecord использует следующие аргументы:

типОбъекта – одна из следующих встроенных констант: acActiveDataObject (активный объект, значение по умолчанию), acDataForm (форма), acDataQuery (запрос), acDataTable (таблица);

имяОбъекта – строковое выражение, представляющее допустимое имя объекта, тип которого указан в аргументе *типОбъекта*.

запись – одна из следующих встроенных констант: acFirst (первая), acGoTo (с указанным номером), acLast (последняя), acNewRec (новая), acNext (следующая) – значение по умолчанию, acPrevious (предыдущая);

смещение – числовое выражение, представляющее число записей, на которое отстоит искомая запись от текущей вперед или назад, если в аргументе *запись* заданы константы acNext или acPrevious, или номер искомой записи, если в аргументе *запись* задана константа acGoTo. Значение выражения должно представлять допустимый номер записи.

Если оставлены пустыми аргументы *типОбъекта* и *имяОбъекта*, то подразумевается активный объект.

Пример перехода к 7 записи:

DoCmd.GoToRecord acDataForm, "Сотрудники", acGoTo, 7

Метод *OpenForm* открывает заданную форму. Метод имеет следующий синтаксис:

DoCmd.OpenForm имяФормы [, режим] [, имяФайла] [, условиеWhere] [, режимДанных] [, режимОкна] [, аргументыОткрытия]

Метод *OpenForm* использует следующие аргументы:

имяФормы – строковое выражение, представляющее допустимое имя формы в текущей базе данных;

режим – одна из следующих встроенных констант: acDesign (конструктор), acFormDS (таблица), acNormal (открытие формы в режиме формы) – значение по умолчанию, acPreview (просмотр);

имяФайла – строковое выражение, представляющее допустимое имя запроса в текущей базе данных;

условиеWhere – строковое выражение, представляющее допустимое предложение SQL WHERE без ключевого слова WHERE;

режимДанных – одна из следующих встроенных констант: acFormAdd (добавление), acFormEdit (изменение), acFormPropertySettings (значения свойств) – значение по умолчанию, acFormReadOnly (только чтение);

режимОкна – одна из следующих встроенных констант: acDialog (окно диалога), acHidden (невидимое), acIcon (значок), acWindowNormal (обычное) – значение по умолчанию;

аргументыОткрытия – строковое выражение, определяющее значение свойства формы OpenArgs. В дальнейшем это значение может быть использовано в программе в модуле формы, например, в процедуре обработки события *Открытие (Open)*. Ссылки на свойство OpenArgs допускаются также в макросах и выражениях.

Предположим, например, что в режиме ленточной формы открывается список авторов. Для того чтобы при открытии формы установить фокус на запись об определенном авторе, достаточно указать фамилию автора в аргументе *аргументыОткрытия*, а затем вызвать метод FindRecord, чтобы переместить фокус на запись для автора с нужной фамилией.

Максимальная длина строки в аргументе условиеWhere составляет 32 768 символов.

В данном примере в форме «Сотрудники», открывающейся в режиме формы, выводятся только записи со значением «Иванов» в поле «Фамилия». Допускается изменение выводимых записей и добавление новых.

DoCmd.OpenForm "Сотрудники", , , "Фамилия = 'Иванов'"

Метод *OpenQuery* позволяет открыть запрос. Метод имеет следующий синтаксис:

DoCmd.OpenQuery имяЗапроса [, режим] [, режимДанных]

Метод *OpenQuery* использует следующие аргументы:

имяЗапроса – строковое выражение, представляющее допустимое имя запроса в текущей базе данных.

режим – одна из следующих встроенных констант: acViewDesign (конструктор), acViewNormal (таблица, значение по умолчанию), acViewPreview (просмотр);

режимДанных – одна из следующих встроенных констант: acAdd (добавление), acEdit (изменение) – значение по умолчанию, acReadOnly (только чтение).

Метод *OpenReport* позволяет открыть отчет. Метод имеет следующий синтаксис:

DoCmd.OpenReport имяОтчета [, режим] [, имяФайла] [, условиеWhere]

Метод *OpenReport* использует следующие аргументы:

имяОтчета – строковое выражение, представляющее допустимое имя отчета в текущей базе данных;

режим – одна из следующих встроенных констант: acViewDesign (конструктор), acViewNormal (печать, значение по умолчанию), acViewPreview (просмотр);

имяФайла – строковое выражение, представляющее допустимое имя запроса в текущей базе данных,

условиеWhere – строковое выражение, представляющее допустимое предложение SQL WHERE без ключевого слова WHERE.

Метод OpenTable позволяет открыть таблицу. Метод имеет синтаксис:

DoCmd.OpenTable имяТаблицы [, режим] [, режимДанных]

Метод *OpenTable* использует следующие аргументы:

имяТаблицы – строковое выражение, представляющее допустимое имя таблицы в текущей базе данных;

режим – одна из следующих встроенных констант: *acViewDesign* (конструктор), *acViewNormal* (таблица – значение по умолчанию), *acViewPreview* (просмотр);

режимДанных – одна из следующих встроенных констант: *acAdd* (добавление), *acEdit* (изменение, значение по умолчанию), *acReadOnly* (только чтение).

Метод RunCommand выполняет команду встроенного меню или встроенной панели инструментов. Синтаксис:

DoCmd.RunCommand команда

Аргумент *команда* является встроенной константой, указывающей, какая именно команда встроенного меню или панели инструментов выполняется. Каждой команде меню или панели инструментов Microsoft Access соответствует константа, которая используется в методе *RunCommand* для выполнения этой команды в программе VBA. Перечень всех констант для аргумента *команда* содержится в разделе «Константы» метода *RunCommand*. Не допускается применение метода *RunCommand* для выполнения команды специального меню или специальной панели инструментов. Он используется только для встроенных меню или панелей инструментов.

Метод SetWarnings позволяет запретить или разрешить вывод сообщений Access. Синтаксис:

DoCmd.SetWarnings режим

Значение *True* (-1) аргумента *режим* задает включение режима вывода системных сообщений, значение *False* (0) отключает вывод сообщений. Режим вывода сообщений, отключенный в программе Visual Basic, останется отключенным даже после прерывания программы нажатием клавиш CTRL+BREAK или после прерывания выполнения программы на точке останова VBA.

Практическая часть

Задание 1. Создание простого модуля VBA

Разработайте процедуру, которая выводит сообщение «Добро пожаловать в автоматизированную систему управления БД» при открытии главной формы.

Задание 2. Автоматизация добавления записи

Создайте форму для ввода данных в таблицу «Сотрудники». Напишите модуль, который проверяет заполнение обязательных полей (например, ФИО, дата рождения) перед добавлением записи.

Задание 3. Фильтрация и поиск данных через модуль

Создайте форму-список с кнопкой «Поиск». Модуль должен фильтровать записи по введённому фрагменту текста (например, по фамилии).

Перечень контрольных вопросов

- 1- Что такое модуль в Access? Какие типы модулей существуют?
- 2- Каким образом осуществляется запуск процедуры VBA в Access?
- 3- Какие события формы наиболее часто используются для автоматизации?
Кратко охарактеризуйте каждое.
- 4- Что такое «валидация данных» и как она реализуется в модуле формы?